AI ASSISTED CODING

LAB-13.1

Name: Rishik

Hall Ticket No: 2303a51778

Batch: 12

Task Description #1 (Refactoring – Removing Code Duplication)

• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

• Instructions:

o Prompt AI to identify duplicate logic and replace it with functions or classes. o

Ensure the refactored code maintains the same output. o Add docstrings to all

functions.

• Sample Legacy Code:

Legacy script with repeated logic print("Area

of Rectangle:", 5 * 10) print("Perimeter of

Rectangle:", 2 * (5 + 10)) print("Area of

Rectangle:", 7 * 12) print("Perimeter of

Rectangle:", 2 * (7 + 12)) print("Area of

Rectangle:", 10 * 15) print("Perimeter of

Rectangle:", 2 * (10 + 15))

• Expected Output:

o Refactored code with a reusable function and no duplication.

PROMPT:

#Identify duplicate logic and replace it with functions or classes, make sure the refactored code maintains the same output.

CODE:

```python
assignment13.1.py > ...
#Identify duplicate logic and replace it with functions or classes, make sure the refactored code maintains the same output.
def calculate_area(length, width):
    "calculates the area of a rectangle given its length and width"
    "input: length and width of the rectangle"
    "output: area of the rectangle"
    return length * width

def calculate_perimeter(length, width):
    "calculates the perimeter of a rectangle given its length and width"
    "input: length and width of the rectangle"
    "output: perimeter of the rectangle"
    return 2 * (length + width)
print("Area of rectangle", calculate_area(5, 10))
print("Perimeter of rectangle ", calculate_perimeter(5, 10))
print("Area of rectangle", calculate_area(7, 12))
print("Perimeter of rectangle ", calculate_perimeter(7, 12))
print("Area of rectangle", calculate_area(10, 15 ))
print("Perimeter of rectangle ", calculate_perimeter(10, 15 ))
```

OUTPUT:

```
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Rishik/One
e/Desktop/AI/assignment13.1.py
Area of rectangle 50
Perimeter of rectangle  30
Area of rectangle 84
Perimeter of rectangle  38
Area of rectangle 150
Perimeter of rectangle  50
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION:

The given program contains repeated blocks of code performing the same calculations (area and perimeter of rectangles). This duplication increases code size and makes maintenance difficult. By identifying common logic, we can create a reusable function and call it with different values. This follows the concept of DRY (Don't Repeat Yourself) and improves code efficiency.

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

• Task: Use AI to analyze a Python script with nested loops and

complex conditionals.

• Instructions: o Ask AI to suggest algorithmic

improvements (e.g., replace nested loops with set

lookups or comprehensions). o Implement changes

while keeping logic intact. o Compare execution time

before and after refactoring.

- Sample Legacy Code: # Legacy inefficient code names

  = ["Alice", "Bob", "Charlie", "David"] search_names =

  ["Charlie", "Eve", "Bob"] for s in search_names:

found = False for

n in names: if s

== n:

found = True

if found:

print(f"{s} is in the list") else:

print(f"{s} is not in the list")

- Expected Output:

o Optimized code using set lookups with performance comparison

table.

PROMPT:

#Refactor this Python code to optimize nested loops: replace inner loops with set lookups or comprehensions while keeping the logic identical. Include a simple timing comparison before and after refactoring.

GIVEN CODE AND OUTPUT:

```python
#Refactor this Python code to optimize nested loops: replace inner loops with set lookups or compre
"""names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
            break
    if found:
        print(f"{s} is in the list")
else:
    print(f"{s} is not in the list")"""
import time
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
# Timing before refactoring
start_time = time.time()
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
            break
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time = time.time()
print(f"Time taken before refactoring: {end_time - start_time:.6f} seconds")
# Refactored code using set lookups
start_time = time.time()
names_set = set(names)  # Convert list to set for O(1) lookups
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time = time.time()
print(f"Time taken after refactoring: {end_time - start_time:.6f} seconds")
```

```
PS C:\Users\katta\OneDrive\Desktop\AIAC> & C:/Users/katt
Charlie is in the list
Eve is not in the list
Bob is in the list
Time taken before refactoring: 0.001000 seconds
Charlie is in the list
Eve is not in the list
Bob is in the list
Time taken after refactoring: 0.000000 seconds
PS C:\Users\katta\OneDrive\Desktop\AIAC>
```

EXPLANATION:

I have understood from this is the program uses **nested loops** to search elements, which increases time complexity. For every element in one list, it checks the entire second list, making it inefficient. We can optimize this by converting the list into a **set**, which allows faster lookups (O(1) time complexity). This improves both performance and readability.

Task Description #3 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:

o Identify repeated or related logic and extract it into reusable functions. o Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

Legacy script with inline repeated logic

price = 250 tax = price * 0.18 total = price

+ tax print("Total Price:", total) price =

500 tax = price * 0.18 total = price + tax

print("Total Price:", total)

- Expected Output:

o Code with a function calculate_total(price) that can be reused for multiple price inputs.

PROMPT:

Refactor this Python code by extracting repeated calculations into a reusable function with docstrings. Make the code modular, readable, and allow multiple inputs without duplicating logic.

CODE:

assignment13.1.py > calculate_total_price

```python
#Refactor this Python code by extracting repeated calculations into a reusable function with docstrings. Make the code
def calculate_total_price(price, quantity):
    """
    Calculate the total price based on unit price and quantity.

    Parameters:
    price (float): The unit price of the item.
    quantity (int): The number of items.

    Returns:
    float: The total price for the given quantity of items.
    """
    tax = price * 0.18
    total_price = price + tax
    return total_price
# Example usage
price = [250, 500]
for p in price:
    total_price = calculate_total_price(p, 1)
    print(f"The total price for an item with unit price {p} is: {total_price}")
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/
Users/Rishik/OneDrive/Desktop/AI/assignment13.1.py
The total price for an item with unit price 250 is: 295.0
The total price for an item with unit price 500 is: 590.0
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION:

The same tax calculation logic is repeated for different inputs. Instead of rewriting the same code, we should extract this logic into a function. This makes the code modular, reusable, and easier to test or update in the future.

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants.
- Instructions:

o Create constants at the top of the file. o Replace all hardcoded

occurrences in calculations with these constants. o Ensure the code

remains functional and is easier to maintain.

- Sample Legacy Code:

Legacy script with hardcoded values print("Area

of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)

- Expected Output: o Code with constants like PI = 3.14159 and RADIUS

= 7 used in calculations.

PROMPT:

Refactor this Python code to replace all hardcoded “magic numbers” with named constants at the top of the file. Ensure calculations remain correct and the code is easier to maintain.

CODE:

```
assignment13.1.py > ...
#Refactor this Python code to replace all hardcoded “magic numbers” with named constants at the top of the file. Ensure calculations remain correct
pi = 3.14159
radius = 7
print("The area of the circle is:", pi * radius ** 2)
print("The circumference of the circle is:", 2 * pi * radius)
```

OUTPUT:

```
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/Local/Python
The area of the circle is: 153.93791
The circumference of the circle is: 43.98226
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION:

From this I have understood the program uses fixed values (like π and radius) directly in calculations. Such values are called magic numbers and reduce code clarity.
By replacing them with named constants, the code becomes more readable, meaningful, and easy to modify.

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

- Task: Use AI to improve readability by renaming unclear variables and adding inline comments.
- Instructions:

o Replace vague names with meaningful ones.

o Add comments where logic is not obvious. o

Keep functionality exactly the same.

- Sample Legacy Code:

Legacy script with poor variable names

a = 10 b = 20 c = a * b / 2 print(c)

- Expected Output: o Code with descriptive variable names like base, height, area_of_triangle, plus explanatory comments.

PROMPT:

#replace vague names with meanningful ones where logic is not obvious and keep functionality exactly the same and add inline comments

CODE:

```
assignment13.1.py > ...
#replace vague names with meaningful ones  where logic is not obvious and keep functionality exactly the same and add inline comments
triangle_base = 10
triangle_height = 20
triangle_area = triangle_base * triangle_height / 2
print("The area of the triangle is:", triangle_area)
```

OUTPUT:

```
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/
The area of the triangle is: 100.0
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION:

The variables used in the program (a, b, c) do not clearly represent their purpose. This makes the code difficult to understand. By using descriptive variable names and adding comments, we can improve readability and make the code self-explanatory.

Task Description #6 (Refactoring – Removing Redundant Conditional Logic) •

Task: Use AI to refactor a Python script that contains repeated if–else logic for

grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function. o Ensure

grading logic remains unchanged.

Code:

marks = 85

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")

marks = 72

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")

• Expected Output:

o A reusable function like calculate_grade(marks) with clean logic and

docstring.

PROMPT:

#Identify redundant conditional checks and replace them with a reusable function and ensure grading logic remains unchanged

CODE:

```
assignment13.1.py > ...
#Identify redundant conditional checks and replace them with a reusable function and ensure grading logic remains unchanged

def calculate_grade(score):
    if score >= 90:
        return "A"
    elif score >= 75:
        return "B"
    else:
        return "C"
marks =85
print(f"Your grade is: {calculate_grade(marks)}")
marks = 72
print(f"Your grade is: {calculate_grade(marks)}")
```

OUTPUT:

```
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData
The area of the triangle is: 100.0
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData
Your grade is: B
Your grade is: C
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION:

The grading logic is repeated multiple times for different inputs. This leads to unnecessary duplication and reduces maintainability. We can refactor this by creating a function for grading, which can be reused for different marks while keeping the logic consistent.

Task Description #7 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.
- Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function. o Improve code readability

without changing behavior.

- Sample Legacy Code: num = int(input("Enter number: "))

square = num * num

print("Square:", square)

- Expected Output:

o Modular code using functions like get_input(), calculate_square(), and

display_result().

PROMPT:

#indentify input, processing, and output section and convert each into a separate function and improve code readability without changing behaviour.

CODE:

```
assignment13.1.py > ...
#indentify input, processing, and output section and convert each into a separate function and improve code readability without changing behaviour.
def get_input():
    return int(input("Enter a number: "))

def calculate_square(num):
    return num * num

def print_ouput(square):
    print("Square of the number is:", square)

num = get_input()
square = calculate_square(num)
print_ouput(square)
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS
Focus folder in explorer (ctrl + click)
PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/AppData/
Enter a number:20
Square of the number is: 400
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION:

The program is written in a procedural style where input, processing, and output are combined. This makes the code less organized.
By dividing the code into separate functions (input, processing, output), we can improve structure, readability, and reusability.

Task Description #8 (Refactoring – Optimizing List Processing)

• Task: Use AI to refactor inefficient list processing logic.

• Instructions:

o Ask AI to replace manual loops with list comprehensions or built-in

functions. o Ensure output remains

identical.

• Sample Legacy Code: nums = [1, 2, 3, 4, 5] squares = [] for n

in nums: squares.append(n * n) print(squares) • Expected

Output: o Optimized version using list comprehension with

improved readability

PROMPT:

Refactor this Python code to use list comprehensions or built-in functions instead of manual loops for list processing. Keep the output identical but make the code more concise and readable.

CODE:

```
assignment13.1.py > ...
# Refactor this Python code to use list comprehensions or built-in functions instead of manual loops for list processing. Keep the output id
"""This code takes a list of numbers and returns a new list containing the squares of those numbers. the original code likely uses a manual
numbers = [1, 2, 3, 4, 5]
squares = [n * n for n in numbers]
print(squares)
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMME

PS C:\Users\Rishik\OneDrive\Desktop\AI> & C:/Users/Rishik/A
[1, 4, 9, 16, 25]
PS C:\Users\Rishik\OneDrive\Desktop\AI>
```

EXPLANATION:

I have understood from this the program uses a loop to generate a new list, which can be simplified. Python provides list comprehension, which is a shorter and more efficient way to process lists. Using list comprehension improves readability and reduces code length.